SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

Code of Conduct:

I P. Lohith Krishna(192325122) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Lohith Krishna

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.

(a) Content-Length: -150 is invalid, because the value must be a non-negative decimal integer (a byte count). (b) Date: Mon, 01 Apr 2026 is inconsistent, because 1 April 2026 falls on a Wednesday, not a Monday, and the HTTP date format requires the correct day name. The status line, Content-Type and Connection fields are syntactically valid, although Content-Type should also declare a charset.

2. Analyze the issue with Content-Length and explain its impact.

Content-Length tells the client exactly how many bytes of body to read. A negative number has no meaning, so the message framing is broken and the client must treat the response as invalid. The impact: the browser may reject the response or show an error page, the page may be truncated or the connection may hang waiting for data, and because of keep-alive the next response on the same connection can be misread (desynchronisation). Proxies and caches may also reject or mis-store such responses, which can lead to security issues such as response splitting or request smuggling.

3. Determine whether the status code matches the response correctness.

No. 200 OK declares that the request succeeded and the response is well-formed, but the header itself is malformed. A server that cannot produce a valid response should send 500 Internal Server Error, and a proxy that receives such a response from upstream should return 502 Bad Gateway.

4. Suggest corrections for the identified errors.

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

Content-Length must equal the exact size of the body in bytes (150 is used here as an example), and the Date must carry the correct weekday. Better still, let the web server generate both fields automatically instead of setting them by hand.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.

The element has no closing tag, so it is left open when appears. In addition, the document has no <!DOCTYPE html>, no lang attribute on <html> and no <meta charset> in <head>.

2. Analyze how the browser will interpret this structure.

HTML parsers are fault-tolerant. When the browser meets , it implicitly closes the open first, so it builds the tree html > head > title and html > body > div > p > text node "HelloWorld". No error is shown to the user. In a strict XML/XHTML parser, however, the mismatched tag is a well-formedness error and parsing stops with an error message.

3. Determine the impact on rendering and layout.

In normal HTML mode the text renders correctly, but the missing DOCTYPE puts the page in quirks mode, so the box model, margins and font-size inheritance can differ between browsers. Relying on auto-correction also makes the DOM unpredictable if more content is added after the , since the paragraph may swallow it or close earlier than intended. This can affect CSS selectors, styling and JavaScript DOM traversal.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
```

```
    </div>
  </body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.

GET appends the data to the URL as a query string (?user=admin&pass=1234), so it travels in the request line and is visible in the address bar. It is limited by URL length and can be bookmarked, cached and repeated safely. POST places the data in the message body (Content-Type: application/x-www-form-urlencoded), so the URL stays clean (/login), there is no practical size limit, and the request is not cached or bookmarked by default. GET is meant for retrieving data; POST is meant for sending data that changes server state.

2. Analyze which method is more suitable for login and why.

POST is more suitable. A login submits sensitive credentials and creates a session (a state change), which is what POST is for. Its data does not appear in the URL, browser history, bookmarks or Referer header, and the response is not cached. Note that POST alone does not encrypt the data, so it must be combined with HTTPS (the "Moderate" level in the table becomes high only with TLS).

3. Identify security risks in using GET for authentication.

The password is exposed in the address bar (shoulder-surfing), saved in browser history and bookmarks, written to web-server, proxy and CDN logs, leaked to third-party sites through the Referer header, and may be cached or shared as a link. A login can also be forged through a simple link or image tag (CSRF), and URL length limits can truncate long passwords.

4. Suggest best practices for secure login implementation.

Use POST over HTTPS/TLS with HSTS. Hash passwords on the server with bcrypt or Argon2 (salted), never storing plain text. Add CSRF tokens, server-side validation and input sanitisation. Apply rate limiting, account lockout and CAPTCHA against brute-force attacks. Use secure session cookies (HttpOnly, Secure, SameSite) and regenerate the session ID after login. Show generic error messages ("Invalid username or password"), avoid logging credentials, and offer multi-factor authentication.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
```

```
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.

</h1> is missing after "Welcome", and is missing after "This is a paragraph". (<img> is a void element, so it needs no closing tag in HTML5.)

2. Analyze issues with the <img> tag.

The tag has no alt attribute, which the HTML standard requires and which screen readers and failed image loads depend on. It also lacks width and height attributes, so the browser cannot reserve space and the layout shifts when the image loads. It sits inside the unclosed <h1>/ as well. In XHTML an image must be self-closed (), whereas HTML5 allows the plain form.

3. Determine how browsers will handle these errors.

Browsers do not stop; they error-correct. A start tag does not close an open <h1>, so the parser nests and <img> inside the heading and closes everything only at </body>. The paragraph and image therefore inherit the heading's large bold style, which the author did not intend. The missing alt is silently accepted, but the page fails accessibility checks and W3C validation.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

Step 1: The browser parses the URL into scheme, host, port and path, and checks its cache and HSTS list.
Step 2: The domain name is resolved to an IP address through the browser/OS cache, then the DNS

resolver. Step 3: A TCP connection (three-way handshake) and, for HTTPS, a TLS handshake are made, then the browser sends GET / HTTP/1.1 with headers such as Host, User-Agent and Cookie. Step 4: The web server routes the request, runs the application logic, queries the database and builds the content. Step 5: The server sends the status line, headers and body (e.g. 200 OK with HTML). Step 6: The browser parses HTML into the DOM and CSS into the CSSOM, builds the render tree, performs layout and paint, fetches extra resources (CSS, JS, images) and runs JavaScript.

2. Identify where delays can occur.

DNS lookup time; TCP and TLS handshake round trips; network latency and limited bandwidth (distance to the server); slow server processing or heavy database queries (high time to first byte); large, uncompressed responses; many sub-requests for CSS, JS and images; and render-blocking scripts or stylesheets in the browser.

3. Analyze the role of DNS in the process.

DNS is the Internet's distributed, hierarchical "phone book". It translates a human-readable domain (www.example.com) into the IP address needed for routing. The resolver queries root, TLD and authoritative name servers, and results are cached according to their TTL, so repeat visits are fast. If DNS fails or is slow, the site appears unreachable even though the server is running. DNS can also direct users to the nearest server (CDN, load balancing), and it is a security target (spoofing, cache poisoning), which DNSSEC helps to prevent.

4. Suggest optimizations to improve performance.

Use DNS caching with a sensible TTL and the dns-prefetch/preconnect hints. Serve content through a CDN. Use HTTP/2 or HTTP/3 with keep-alive and TLS 1.3 to cut handshake cost. Enable gzip/Brotli compression and browser caching headers (Cache-Control, ETag), and minify CSS/JS. Optimise and lazy-load images, load scripts with async/defer, and reduce the number of requests. On the server, add database indexes and server-side caching.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand